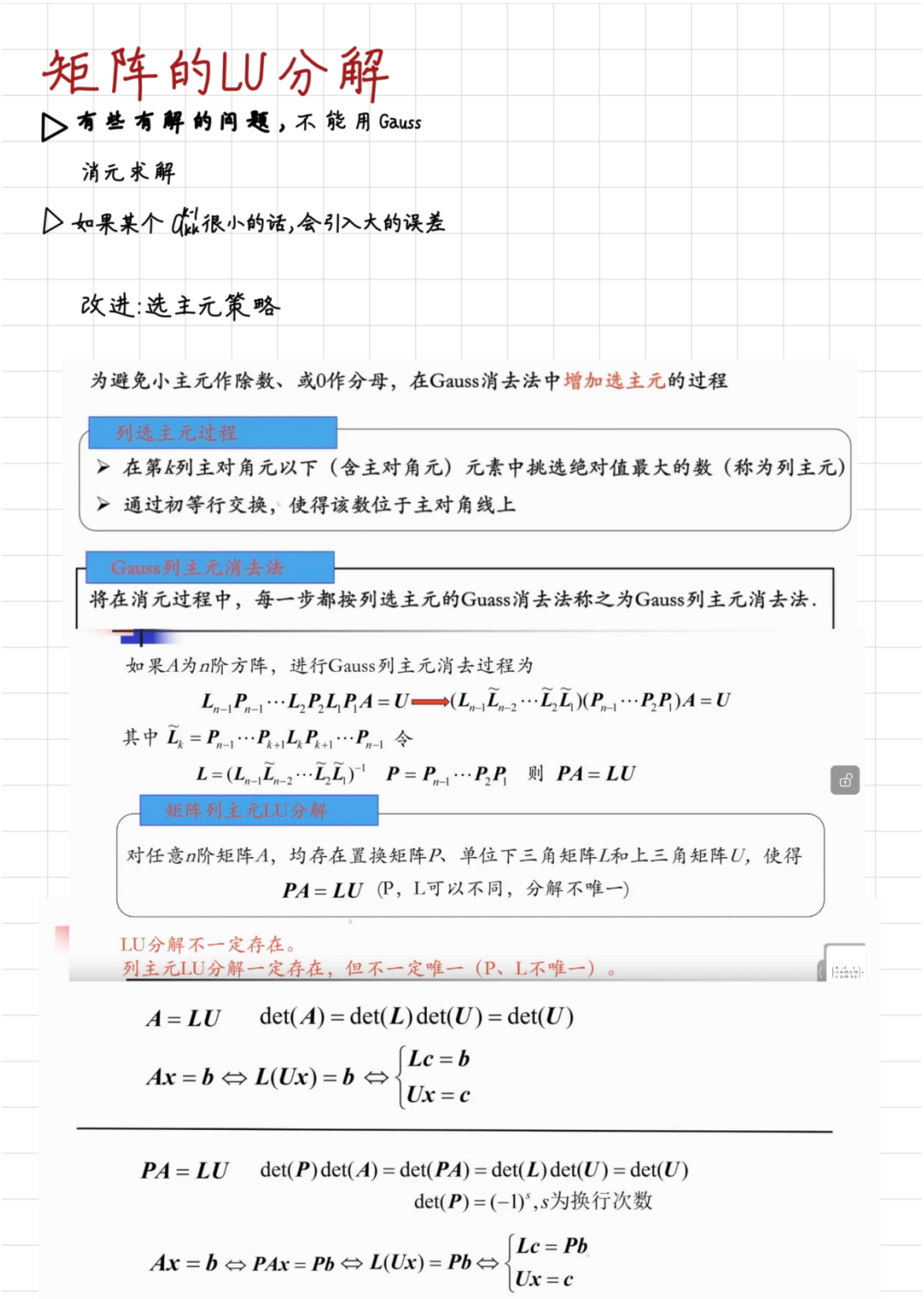
算法作业五

2025 年 10 月 20 日

T1 调研学习并给出矩阵的 LU 分解方法

1 问题分析（学习笔记）



2 算法思路

1. 初始化：L 为单位下三角矩阵，U 为原始矩阵副本。
2. 选主元：每列中选择绝对值最大的元素作为主元，交换行。
3. 消元：

- 更新 L 矩阵的非对角元素。
- 更新 U 矩阵的当前行以下元素。

3 C 语言实现代码（Doolittle+部分选主元）

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
 #define N 3
void printMatrix(double arr[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%.4lf\t", arr[i][j]);
        }
        printf("\n");
    }
}
void swap(double arr[N][N],int row1,int row2) {//交换两行
    for (int i = 0; i < N; i++) {
        double temp = arr[row1][i];
        arr[row1][i] = arr[row2][i];
        arr[row2][i] = temp;
    }
}
int LUcomp(double A[N][N], double L[N][N], double U[N][N], int pivot[N]) {
    for (int i = 0; i < N; i++) {
        pivot[i] = i;
    }
    for (int i = 0; i < N; i++) {//列
        int pivotrow = i;//主元
        for (int j = i + 1; j < N; j++) {//行
            if (fabs(A[j][i]) > fabs(A[pivotrow][i])) {
                pivotrow = j;
            }
        }
        if (pivotrow != i) {
            swap(A, pivotrow, i);//交换两行
            int temp = pivot[i];//记录置换
            pivot[i] = pivot[pivotrow];
            pivot[pivotrow] = temp;
        }
        if (A[i][i] == 0) {
            return 0;
        }
```

```
40              for (int j = i + 1; j < N; j++) {//行
41                  L[j][i] = A[j][i] / A[i][i];
42                  for (int k = i; k < N; k++) {
43                      A[j][k] = A[j][k] - A[i][k] * L[j][i];
44                  }
45              }
46          }
47          for (int m = 0; m < N; m++) {//构造L和U矩阵
48              for (int n = 0; n < N; n++) {
49                  if (n > m) {
50                      U[m][n] = A[m][n];
51                      L[m][n] = 0.0;
52                  }
53                  else if (n == m) {
54                      U[m][n] = A[m][n];
55                      L[m][n] = 1.0;
56                  }
57                  else {
58                      U[m][n] = 0;
59                  }
60              }
61          }
62          return 1;
63      }
64      int main() {
65          double A[N][N] = { {2,-1,1},{4,1,-1},{1,1,1} };
66          double L[N][N] = { 0 },U[3][3] = { 0 };
67          int pivot[N];
68          if (LUcomp(A, L, U, pivot)) {
69              printf("L的矩阵为：\n");
70              printMatrix(L);
71              printf("U的矩阵为：\n");
72              printMatrix(U);
73          }
74          else {
75              printf("矩阵不可逆");
76          }
77          return 0;
78      }
```

4 运行示例

```
L的矩阵为:
1.0000  0.0000  0.0000
0.5000  1.0000  0.0000
0.2500  -0.5000 1.0000
U的矩阵为:
4.0000  1.0000  -1.0000
0.0000  -1.5000 1.5000
0.0000  0.0000  2.0000
```

5 性能分析

指标	高斯消元法	LU分解
时间复杂度	$O(n^3)$（单次求解）	$O(n^3)$（分解） + $O(n^2)$（单次求解）
空间复杂度	$O(n^2)$（存储增广矩阵）	$O(n^2)$（存储L和U矩阵）
重复求解效率	每次需重新计算（$O(n^3)$）	分解一次后，多次求解仅需$O(n^2)$

T2：给出方案计算可逆矩阵的逆

1 问题分析

给定一个可逆矩阵 $A \in R^{n\times n}$，求它的逆矩阵 A^{-1}，满足：

$$AA^{-1} = A^{-1}A = I$$

2 算法思路

我们可以通过解 n 个具有相同系数矩阵 A 的线性方程组来求出这些未知数，未知数向量 x^j 是逆矩阵的第 j 列，右边的向量 e^j 是单位矩阵的第 j 列($1 \leqslant j \leqslant n$)：

$$Ax^j = e^j$$

通过 n 阶单位矩阵得到 A 的扩展矩阵以后，我们可以应用高斯消去法来解这些方程组。但更好的做法是，我们可以使用高斯消去法求出 A 的 LU 分解，然后就像我们前面解释过的，再对方程组 $LUx^j = e^j$，$j = 1, \cdots, n$ 求解。

$$A = LU \quad \Rightarrow \quad AX = I \Rightarrow LUX = I$$

对每一列单位向量 e^j，解两步方程：

1. $Ly = e^j$
2. $Ux^j = y$

优点：重复求解高效。缺点：需先分解。

前向替代公式（第 i 行）：

$$y_i = e_i - \sum_{j=0}^{i-1} L_{i,j}y_j$$

后向替代公式：

$$x_i = \frac{y_i - \sum_{j=i+1}^{N-1} U_{i,j}x_j}{U_{i,i}}$$

最后把列向量 x 写入 inv[:,col]。

3 C 语言运行代码（LU 分解）

LU 分解函数代码同上，以下为矩阵求逆的代码：

```c
#include "LU.h"
int inverseLU(double* A[N][N], double inv[N][N]) {
    double L[N][N] = { 0 }, U[N][N] = { 0 };
    int pivot[N];
    LUcomp(A, L, U, pivot);
    for (int col = 0; col < N; col++) {
        double y[N] = { 0 }, x[N] = { 0 }, e[N] = { 0 };
        e[col] = 1;
        for (int i = 0; i < N; i++) {//Ly=E, 解y
            double sum = 0;
            for (int j = 0; j < i; j++) {
                sum += L[i][j] * y[j];
            }
            y[i] = e[i] - sum;
        }
        for (int i = N - 1; i >= 0; i--) {//Ux=y,解x
            double sum = 0;
            for (int j = i + 1; j < N; j++) {
                sum += U[i][j] * x[j];
            }
            x[i] = (y[i] - sum) / U[i][i];//把列向量x代入
        }
        for (int i = 0; i < N; i++) {
            inv[i][col] = x[i];
        }
    }
}
int main() {
    double A[N][N] = { {2,-1,0},{1,2,1},{3,0,1} };
    double inv[N][N];
    printf("LU 分解法 A ^ -1\n");
    if (inverseLU(A, inv))
        printMatrix(inv);
    else
        printf("矩阵不可逆");
    return 0;
}
```

4 运行示例

```
LU 分解法 A ^ -1
-0.5000 0.5000  1.0000
-1.0000 1.0000  1.0000
2.5000  -1.5000 -3.0000
```

5 性能分析

对比维度	LU 分解	高斯消去法（求逆）
时间复杂度（n 阶矩阵）	分解阶段：$O(n^3)$； 单次求解：$O(n^2)$	全程 $O(n^3)$（无分解 / 求解分离）
空间复杂度	$O(n^2)$（需存储 L、U）	$O(n^2)$（需存储 n×2n 增广矩阵）